

EdgeGen: Efficient LLM-Empowered Model Generation with Quantization-Aware NAS

Yingqi Peng
Zhejiang Key Lab. of Accessible
Perception and Intelligent Systems,
College of Computer Science,
Zhejiang University
Hangzhou, Zhejiang, China
yingqipeng@zju.edu.cn

Wenhao Zhou
Zhejiang Key Lab. of Accessible
Perception and Intelligent Systems,
College of Computer Science,
Zhejiang University
Hangzhou, Zhejiang, China
zwenhao@zju.edu.cn

Kaijie Gong
Zhejiang Key Lab. of Accessible
Perception and Intelligent Systems,
College of Computer Science,
Zhejiang University
Hangzhou, Zhejiang, China
gongkj@zju.edu.cn

Yang Liu
Zhejiang Key Lab. of Accessible
Perception and Intelligent Systems,
College of Computer Science,
Zhejiang University
Hangzhou, Zhejiang, China
quakso@zju.edu.cn

Yi Gao
Zhejiang Key Lab. of Accessible
Perception and Intelligent Systems,
College of Computer Science,
Zhejiang University
Hangzhou, Zhejiang, China
gaoyi@zju.edu.cn

Wei Dong
Zhejiang Key Lab. of Accessible
Perception and Intelligent Systems,
College of Computer Science,
Zhejiang University
Hangzhou, Zhejiang, China
dongw@zju.edu.cn

Abstract

The rapid evolution of the Web of Things (WoT) has created new opportunities for connectivity and standardization across heterogeneous devices, enabling the development of increasingly complex systems. However, edge devices deployed in resource-constrained scenarios face significant challenges. These devices require lightweight and efficient models to achieve high accuracy while operating within strict memory constraints. Typical approaches to model generation, Neural Architecture Search (NAS), have proven effective in automating the search for optimal architectures. However, existing NAS methods suffer from two critical limitations: (1) they fail to incorporate quantization into the search space, which can result in overlooking larger models that might perform better after quantization; and (2) current model evaluation methods struggle to provide accurate assessments within a short time period. To address these challenges, we propose EdgeGen, a novel NAS framework that integrates multiple quantization methods into the search process, enabling the discovery of larger models that need quantization to satisfy the constraint. EdgeGen employs a multi-beam Monte Carlo Tree Search (MCTS) algorithm and a constraint validator to explore the expanded search space efficiently, searching vast original and quantized models. Furthermore, EdgeGen evaluates the model performance following a GNN-based performance predictor, which provides a rapid and precise prediction. Across multiple benchmarks, EdgeGen consistently outperforms state-of-the-art NAS methods. Code available at: <https://doi.org/10.5281/zenodo.18323232>.

CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Computer systems organization** → **Embedded software**.

Keywords

NAS, AI agent, MCTS, Performance Predictor, Quantization

ACM Reference Format:

Yingqi Peng, Wenhao Zhou, Kaijie Gong, Yang Liu, Yi Gao, and Wei Dong. 2026. EdgeGen: Efficient LLM-Empowered Model Generation with Quantization-Aware NAS. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792394>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18323232>.

1 Introduction

The rapid growth of the Web of Things (WoT) has made it easier to connect and standardize heterogeneous devices, allowing for the creation of more complex systems. As WoT evolves, edge devices are required to handle diverse and heterogeneous tasks while operating under strict resource constraints. Among various tasks deployed on edge devices, Human Activity Recognition (HAR) stands out as a prominent and computationally demanding one. HAR provides critical context for health monitoring [1] and user habit studies [12], offering greater energy efficiency and privacy protection compared to vision-based alternatives [14]. But how to dynamically generate models for HAR tasks on edge devices remains a challenging problem.

To address the need for such models, a general approach that can be used is Neural Architecture Search (NAS). NAS is a method that can automatically search for the required models from a search space containing billions of possibilities. NAS has numerous research efforts, and it can be broadly divided into two categories:



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792394>

traditional NAS and LLM-empowered NAS. Traditional NAS approaches include reinforcement learning methods, differentiable methods, and Super Network methods [10, 15, 20]. Among them, Super Network is the most widely used approach for research on resource-constrained model generation [4, 6, 13]. The Super Network can quickly generate model weights to evaluate the model's accuracy, helping NAS efficiently identify the desired model. However, the training overhead of Super Networks is extraordinarily high. For instance, the MIT team utilized 32 V100 GPUs for distributed training, requiring a total of 1200 GPU hours [3]. Moreover, the training overhead of a Super Network with quantization is even greater than that of a standard Super Network.

LLM-empowered NAS leverages LLM for the search process. Due to the extensive knowledge base of LLM, they can identify relatively optimal models during the search process without training LLM. Since there is no limitation of training cost, many NAS researches have combined LLM in recent years [5, 7, 11, 16, 17, 19].

Nevertheless, these LLM-empowered NAS suffer from two key drawbacks: (1) *They fail to incorporate quantization into the search space.* These methods directly search for models based on the task's constraints, skipping over models that do not meet the constraints. These approaches may lead to overlooking models that should have been considered. For example, a larger model might exceed the constraints initially, but after quantization, it could meet the constraints and outperform most smaller models. (2) *They lack a model evaluation method that is both time-efficient and highly accurate.* Current methods for model evaluation in LLM-empowered NAS can be classified into two types: training-based methods and zero-cost proxies. Training-based methods face a trade-off between time and accuracy. Fully training models wastes time on unpromising candidates, while limited training may underestimate promising models due to varying convergence speeds. On the other hand, zero-cost proxies are faster but suffer from low accuracy, with Kendall coefficients may below 0.2 (shown in Table 9 - Table 13). Both approaches have significant limitations, highlighting the need for more efficient and reliable evaluation methods.

The reason these methods struggle to overcome these two drawbacks is that (1) incorporating quantization significantly expands the search space. Since quantization allows larger models to be included in the search, even adding just one additional stage can inflate the search space by up to 34,560 times (as detailed in Section 3). (2) The performance of models is difficult to predict, particularly for quantized models. Because predicting model performance is influenced by both the task and the model's own characteristics, and different quantization methods can yield drastically different performance degradation across various models and tasks. Moreover, many zero cost proxies are not designed for quantization, making it difficult for them to effectively evaluate the performance of quantized models (shown in Figure 6).

To address these issues, we aim to design a novel NAS framework incorporating LLM that: (1) expands the search space of LLM by integrating Quantization Aware Training (QAT) and Static Post Training quantization (PTQ), (2) employs a constraint validator to evaluate whether a original or quantized model meets the given constraints, and (3) utilizes a performance predictor to assess model performance during the search process, thereby reducing training overhead. In this paper, we propose EdgeGen, an edge-oriented

model generation framework. To fully explore the search space and exploit the previously discovered models, we represent the search process as a tree and employ multi-beam Monte Carlo Tree Search (MCTS). Besides, we use a constraint validator to check the original or quantized candidates whether satisfy the constraint. Moreover, we generate 16,823 samples for training the performance predictor, which estimates the model's performance to achieve fast and accurate prediction results.

To summarize, our contributions are three-fold:

(1) We design an EdgeGen framework that integrates quantization. This framework can search in a quantization-aware search space. (2) We generate 16,823 samples to train a GNN-based performance predictor that can predict the performance of original, QAT, and static PTQ models within 0.5 seconds. (3) We conduct a series of experiments to validate the performance of our framework. We surpass the state-of-the-art (SOTA) on multiple benchmarks.

2 Related Work

2.1 Traditional NAS

Early NAS methods primarily relied on reinforcement learning or evolutionary algorithms to search for individual architectures case by case, resulting in extremely high computational costs. Once-for-All (OFA) [3] was the first to propose the concept of a "Super Network", which trains a shared-weight network that supports multiple sub-network architectures. This decouples training from searching, significantly reducing the cost of multi-scenario deployment.

After the OFA proposed the Super Network paradigm, the research community began to extensively explore how to integrate super network with model quantization to further compress models and adapt them to resource-constrained edge devices. Bigger&Faster [4] introduced a quantization-aware two-stage NAS method that searches for 8-bit integer quantized Transformers based on super network. Under various constraints, it significantly outperformed the SOTA AutoTinyBERT [2]. HQNAS [6] proposed a hardware-aware quantized NAS framework, which uses weight-sharing and bit-sharing super network mechanisms to jointly search for architectures and quantization strategies within less GPU time (e.g., only about 4 hours on CIFAR-10). These works can get high-precision models that meet the environmental constraints based on super networks, but they all need to spend a lot of computational overhead on super network training.

2.2 LLM-empowered NAS

In recent years, breakthroughs in LLMs for code generation and reasoning tasks have provided new ideas for NAS. For example, Evo-Prompting [5] leverages LLM as adaptive crossover and mutation operators in evolutionary algorithms to directly generate network architectures in the form of code. Without the need for a super network, it reduces NAS's reliance on computational resources and enhancing the flexibility of the search process. LAPT [19] introduces a new paradigm of design principle transfer. This method utilizes LLMs to learn design principles from existing architectures and then transfers these principles to optimize the search space for new tasks. This approach significantly improves search efficiency, reducing search time by 5 to 15 times. The RZ-NAS [7] method

achieves efficient neural architecture search by combining the text and code understanding capabilities of LLM with zero cost proxy evaluation strategies. This method leverages the reflective module of LLM to generate architecture mutations and provide language-based guidance, while employing SOTA evaluation methods such as Zico [9] and Zen_Score [8] to assess the performance of candidate architectures. As a result, this approach demonstrates superior performance across multiple benchmarks. However, these works

Table 1: Neural architecture search space configuration.

Category	Parameter	Options
Structural	Stages	{1, 2, 3, 4}
	Blocks per stage	{1, 2}
	Skip connection	{True, False}
	Quantization mode	{None, Static, QAT}
Convolution	Conv type	{SeDpConv, MBConv, DpConv, SeSepConv, DWSepConv}
	Kernel size	{3, 5, 7}
	Stride	{1, 2, 4}
Channel	Base channels	{8, 16, 24, 32}
	Expansion ratio	{1, 2, 3, 4}
	SE module	{True, False}
	SE ratio	{0, 0.25, 0.5}
Activation	Type	{ReLU6, LeakyReLU, Swish}

only directly search for models that meet the requirements without incorporating quantization-aware NAS. Since a larger model, after quantization, may still achieve better performance than a smaller model. So NAS without considering quantization might miss some efficient or even optimal models.

3 Preliminaries

Problem Definition Under the given constraints (e.g., *memory* $\leq$ *max_memory*), use an algorithm to search for models with the highest accuracy.

Modeling Definition Utilize search algorithms, LLM, constraint validator and performance predictors to explore optimal model architectures and store them using a tree structure. Each node represents a model architecture, and the edge from node n_i to node n_j indicates that n_i is the parent node of n_j .

Generator Definition The LLM serving as the model generator is denoted as Φ . During each model architecture generation process, the LLM generates model architectures within the search space based on information such as parent nodes, failure data, and experiential feedback.

Search Space We specifically design our own quantization-aware search space, shown in Table 1. Our search space is enormous, with each individual stage having a configuration size of Per Stage Search Space = $|Blocks| \cdot |Connection| \cdot |Conv| \cdot |KernelSize| \cdot |Stride| \cdot |BaseChannels| \cdot |ExpansionRatio| \cdot |SERatio| \cdot |Activation| = 25920$. When only considering 4-stage search space, the total search space expands to

$$|Quantizationmode| * (25920)^4 = 1.35 \times 10^{18}.$$

Although there are some constraint relationships between different parameters that may reduce the search space, this calculation only considers the 4-stage configurations. With 1,2,3-stage configurations not yet included, the actual search space size undoubtedly exceeds the tens of billions level.

4 Methodology

EdgeGen consists of a multi-beam MCTS algorithm and two components: a validator and predictor. The multi-beam MCTS algorithm runs throughout the entire search process, as shown in Figure 1. It encompasses how nodes are selected, how nodes are expanded, and how the validator and predictor are utilized.

4.1 Multi-Beam MCTS

Define each generated long-term model architecture as a node n_i , where $n_i \in N$ and $N = \{n_i | i = 1, 2, 3, \dots\}$. These nodes are all selected by the algorithm and satisfy the given constraints. Each of them has a corresponding score s_i , which represents the accuracy achieved on the data set after being trained for $\mathcal{E}$ epochs.

In each iteration, the algorithm selects a node based on the soft mixed probability and use this node as the parent node for expansion.

$$P_{\text{mixed}}(i) = \lambda \cdot \frac{1}{k} + (1 - \lambda) \cdot \frac{e^{\alpha * (s_i - s_{\max})}}{\sum_{j=1}^k e^{\alpha * (s_j - s_{\max})}} \quad (1)$$

$$P(i) = \frac{P_{\text{mixed}}(i)}{\sum_{j=1}^k P_{\text{mixed}}(j)} \quad (2)$$

In Equation 1, k denotes the top k nodes. To ensure exploration, if the root node is not included among the top k nodes, it will be added. After obtaining the probability $P(i)$ for each node from Equations 1-2, P is used as the selection probability for each node in a random selection function (equation 3), which outputs one node as the selected node (node 2 is the selected in Figure 1).

$$i_{\text{selected}} \leftarrow \text{random}(k, \text{probabilities} = P) \quad (3)$$

After the selected node $n_{i_{\text{selected}}}$ is obtained, its information will be incorporated into the prompt. Then Φ generates M expanded nodes based on the selected node. All extended nodes are recorded as candidate nodes $C = \{C_1, C_2, \dots, C_{m-1}, C_m\}$. Subsequently, a constraint validator $\mathcal{V}$ is used to check whether the candidate node meets the constraints. If a candidate node C_i fails the validation, its information is added to the session failure nodes, and the candidate node will no longer remain active, denoting it as $C_i \notin \mathcal{A}$. On the other hand, candidate nodes that pass the validation will be considered active nodes and will be recorded, denoting it as $C_i \in \mathcal{A}$.

In each iteration, Φ performs k attempts. The next step proceeds only if the proportion of active nodes among the M candidate nodes in an attempt is $\geq (60\% * M)$. Otherwise, the process moves to the next attempt, and the failure information from the previous attempt is added to the session failure nodes, which are then used as prompts for Φ .

The candidate nodes that pass $\mathcal{V}$ and meet the required pass rate are used as inputs to the performance predictor $\mathcal{P}$. This node that has the best predicted accuracy $s_i^{\mathcal{P}}$ is saved as a long-term node n_i . If none of the k attempts achieves the required pass rate $\geq (60\% * M)$, the algorithm will select the node with the highest $s_i^{\mathcal{P}}$.

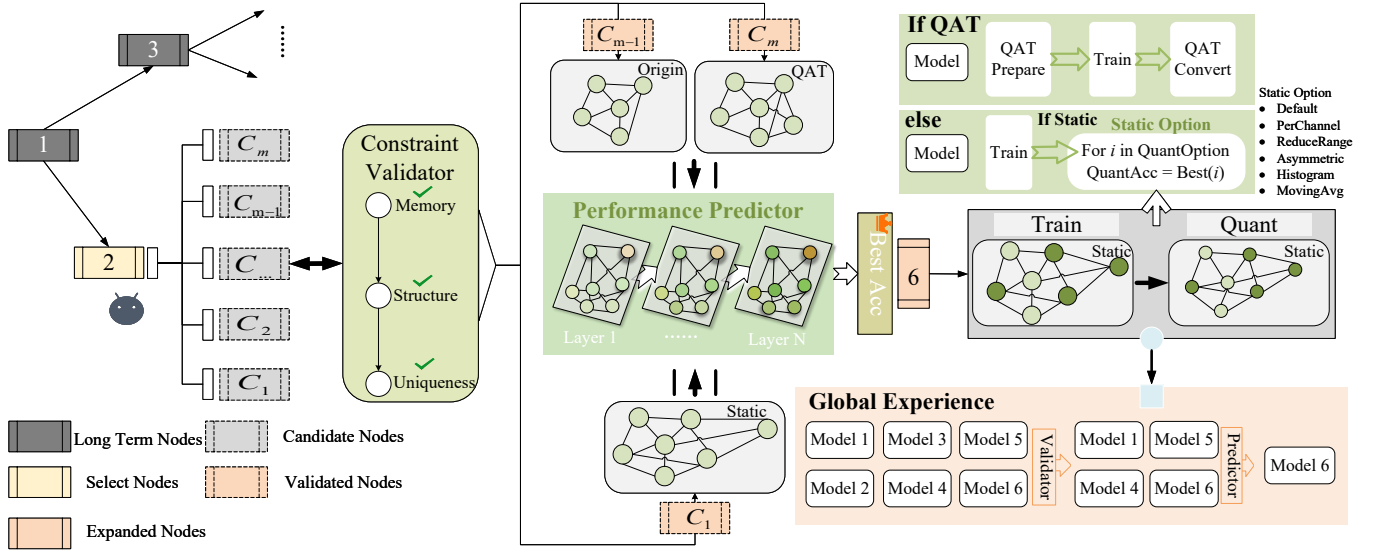


Figure 1: Overview of the multi-beam MCTS. The algorithm uses the equations to compute the selected node, then Φ generates M candidate nodes. These candidates are passed through a constraint validator, and validated nodes enter the performance predictor. The best node is trained and may quantized. Finally, EdgeGen updates the experience.

from the remaining nodes that pass $\mathcal{V}$. If the model is quantized, it will be processed according to its quantization mode (Static or QAT). As shown in Figure 1, the Static mode will evaluate six methods after training and select the best one, while the QAT mode will be processed before training begins.

The algorithm repeats the above process until the maximum time $\mathcal{T}$ is reached.

4.2 Constraint Validator

The constraint validator is designed to validate candidate architectures against a set of constraints, ensuring that the generated models are feasible for deployment and meet constraints. It verifies structural integrity, structural validity and memory usage, while also providing actionable suggestions.

When the constraint validator gets a model architecture, it first checks the structural integrity. This mainly verifies that configuration parameters are completely defined, including input channels, output channels, stride, kernel size and so on. Then it checks the validity of the model, which can figure out models that cannot be reasoned and trained in advance. For example, the output channel of SeDpConv must be equal to the input channel, otherwise an error will be reported. However, such model structure problems are easy to be ignored by the generator Φ . Therefore, the validator generates relevant violations and suggestions to prompt the generator to modify.

Subsequently the validator calculates the memory usage of the candidate model by leveraging the calculate memory usage function. This function computes both activation memory and parameter memory by one forward pass. During a forward pass with a input tensor, the function tracks the size of output tensors. The memory consumed by activation is calculated using the number of elements in the output tensor (numel) multiplied by the size of each element

Algorithm 1 EdgeGen Algorithm: Multi-beam MCTS with Constraint Validation and Performance Prediction

Input: Maximum time $\mathcal{T}$, expansion factor M , top- k parameter, training epochs $\mathcal{E}$;

Output: The long term nodes N ;

```

1:  $t \leftarrow 0$ ;
2: while  $t \leq \mathcal{T}$  do
3:   Compute probabilities  $P(i)$  using Equations (1)-(2);
4:   Select node  $n_{i_{\text{selected}}}$  using Equation (3);
5:    $\Phi$  Generate  $M$  candidate nodes  $C = \{C_1, C_2, \dots, C_M\}$  based
   on  $n_{i_{\text{selected}}}$ 
6:   for each candidate node  $C_i \in C$  do
7:     if  $\mathcal{V}(C_i)$  succeeds then
8:       Mark  $C_i$  as active:  $C_i \in \mathcal{A}$ 
9:     else
10:      Mark  $C_i$  as inactive:  $C_i \notin \mathcal{A}$ 
11:      Add  $C_i$  to session failure nodes
12:   end if
13:   end for
14:   if  $\frac{|\mathcal{C}_i \in \mathcal{A}|}{M} \geq 60\%$  then
15:     Use active nodes  $C_i \in \mathcal{A}$  as inputs to  $\mathcal{P}$ 
16:     Select node  $n_i$  with highest predicted accuracy  $s_i^{\mathcal{P}}$ 
17:     Save  $n_i$  as long-term node
18:   else
19:     Move to next attempt and update session failure nodes
20:   end if
21:   Update the variables and experience;
22: end while

```

(element size), this is $\text{numel} * \text{size}$, which shown in Figure 2. Similarly, the memory consumed by model parameters is computed

by iterating through all parameters in the model and calculating $\text{numel} * \text{size}$. Moreover, this calculation method can also accurately estimate the size of quantized models. Therefore, larger models that can meet memory constraints through quantization will not be skipped. The validator also generates memory violations and suggestions to help Φ modify.

Finally, the validator checks the duplicate of the model architecture. It determines whether the model is repeated by traversing the long-term nodes that have been saved before, and generates repeated violations to prompt the generator.

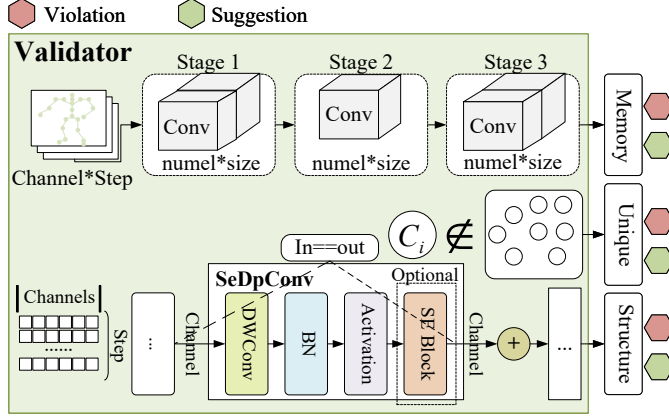


Figure 2: Schematic of the constraint validator $\mathcal{V}$.

4.3 Performance Predictor

To enable accurate prediction of model performance across different modes including origin, static PTQ and QAT, we design a graph-based neural network (GNN) predictor. This predictor represents the architecture of neural networks as graph structures, capturing both local and global dependencies within the architecture. By leveraging this representation, the predictor provides precise performance estimates for various modes.

4.3.1 Graph-Based Modeling of Neural Architectures. The predictor utilizes a graph convolutional network (GCN) to encode the structure and properties. Each architecture is represented as a directed graph, where nodes correspond to convolution blocks, and edges represent the flow of data between blocks. The encoder converts each block attributes (such as kernel size, stride, activation function, and the existence of SE block and so on) into one-hot vectors, while continuous attributes (such as the number of channels) are normalized to ensure consistent scaling. Additionally, input and output nodes are added to the graph to represent the input data and the global pooling/classification layer, respectively. As shown in Figure 3, the model with five blocks is transformed into a graph representation. Each block becomes a graph node, and the edge represents their connection relationship, and two nodes are added at the beginning and the end to represent the input and output.

4.3.2 Graph Neural Network Design. As illustrated in Figure 3, the GNN predictor uses multiple layers of graph attention convolution (GATConv) to extract structural information from the input graph.

First, the node features are projected into a hidden representation using a linear layer. The graph is then processed through several GATConv layers, where each layer updates the node embeddings by aggregating information from neighboring nodes. Batch normalization is applied after each GATConv layer to stabilize training, followed by a ReLU activation and dropout for regularization. After the convolutional layers, global mean pooling aggregates the node embeddings into a single vector representing the entire architecture. This vector is passed to three separate fully connected layers, each predicting the model's accuracy for origin, static PTQ, and QAT modes.

4.3.3 Training the Predictor. To train the predictor, we generated a data set of 16,823 neural network architectures, each is evaluated under three modes: origin, static PTQ, and QAT. For each architecture, we record its accuracy across these modes, creating a well-structured corpus that assists predictor to capture the relationship between accuracy and model architecture. The predictor is trained to minimize the mean squared error (MSE) between its predictions and the ground truth accuracy, ensuring that it could predict the performance of unseen architectures accurately.

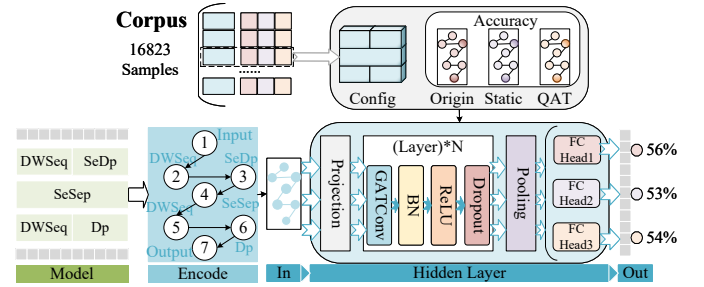


Figure 3: Schematic of the performance predictor $\mathcal{P}$.

5 Experiments

5.1 Experimental Setup

5.1.1 Benchmarks. HAR tasks are among the most common applications of the WoT in resource-constrained scenarios. These tasks are highly dynamic and require various heterogeneous models to execute. Moreover, HAR tasks are time-series multi-classification problems, making them more challenging than typical tasks. Therefore, we select the five most challenging and representative benchmarks from 18 HAR benchmarks to conduct our experiments (see Appendix A for details).

5.1.2 Constraint. Due to the differences in time steps and channels across benchmarks (shown in Table 6), the size of each sample varies. Based on the characteristics of the benchmarks, we categorize Mhealth and USCHAD as benchmarks with larger memory constraints, while MMAAct, UTD-MHAD, and Wharf are categorized as benchmarks with smaller memory constraints. Therefore, we set different memory constraints for these two categories of benchmarks.

Table 2: Comparison of accuracy across benchmarks and methods under different memory constraints.

Benchmark	Constraint	EdgeGen		LAPT		LAPT (Quant)		RZ-NAS		RZ-NAS (Quant)		UniMTS
		Accuracy	Mode	Accuracy	Mode	Accuracy	Mode	Accuracy	Mode	Accuracy	Mode	
Mhealth	12MB	89.63%	QAT	81.10%	Origin	80.48%	Static	82.32%	Origin	84.76%	QAT	97.00% >200MB
	15MB	96.95%	QAT	78.05%	Origin	88.41%	Static	92.07%	Origin	74.39%	QAT	
	20MB	99.39%	Static	84.76%	Origin	92.68%	Static	89.63%	Origin	96.95%	QAT	
MMAct	3MB	61.23%	Static	52.35%	Origin	58.17%	Static	52.40%	Origin	54.81%	Static	77.10% >200MB
	5MB	62.07%	Static	54.59%	Origin	59.56%	Origin	53.21%	Origin	59.19%	Static	
	8MB	63.60%	Origin	62.87%	Origin	56.84%	Origin	59.67%	Origin	59.78%	Static	
UTD-MHAD	3MB	55.58%	Origin	43.26%	Origin	16.74%	Static	42.79%	Origin	21.40%	Static	85.60% >200MB
	5MB	59.53%	Origin	53.95%	Origin	15.35%	Static	42.33%	Origin	46.98%	Origin	
	8MB	69.84%	Origin	61.40%	Origin	64.65%	Origin	59.53%	Origin	57.67%	Origin	
Wharf	3MB	67.73%	Static	55.00%	Origin	59.09%	Static	51.82%	Origin	47.27%	Origin	82.70% >200MB
	5MB	67.05%	Static	59.09%	Origin	62.73%	Static	57.73%	Origin	42.73%	Static	
	8MB	70.23%	Static	65.00%	Origin	55.45%	Static	60.91%	Origin	60.91%	Static	
USCHAD	12MB	72.35%	Origin	72.11%	Origin	64.48%	Origin	70.44%	Origin	69.25%	Origin	58.30% >200MB
	15MB	72.71%	Origin	70.80%	Origin	62.10%	Static	68.06%	Origin	44.93%	QAT	
	25MB	76.47%	Origin	74.14%	Origin	73.42%	Origin	69.61%	Origin	69.73%	Origin	

5.1.3 Baselines. Since we design the algorithm of LLM-empowered NAS, we compare it with the SOTA algorithm of LLM-empowered NAS. Here are our baselines.

- **LAPT (AAAI'24).** LAPT is to search for the optimal architecture by learning the general principles of model design. It is the SOTA in Transferable Neural Architecture Search (TNAS) and uses the training to predict the performance.
- **LAPT (Quantization).** To compare LAPT's performance in the quantized search space, we enhance it with a constraint validator, enabling it to identify more models for quantization.
- **RZ-NAS (ICML'25).** RZ-NAS designs a LLM-based evolutionary algorithm and uses the zero cost proxy method to evaluate the performance of the model. It is SOTA in the current combination of NAS and LLM.
- **RZ-NAS (Quantization).** Similarly, we enhance it with a constraint validator, enabling it to identify more models for quantization.
- **UniMTS (NeurIPS'24) [18].** UniMTS is an expert model specifically designed for HAR tasks, with a memory footprint exceeding 200MB, representing the handcrafted SOTA in customized HAR models.

5.1.4 Metrics.

- **Accuracy:** The accurate rate of the model on the test set, which represents its performance.
- **Top-K Hit Rate:** Evaluates the proportion of correctly predicted models within the top K positions.
- **Kendall τ :** Measures the ordinal correlation between predicted model ranking and true model ranking based on pairwise concordance.

We use ChatGPT-4o as the foundation model and set the time limit 1 hour. Besides, we conduct multiple runs of the experiments and report the average results.

5.2 Main Results

As shown in Table 2, our algorithm outperforms the current SOTA methods. We use pink, violet, and orange to represent the three modes: Origin, Static, and QAT.

From the experimental results, our method outperforms the baselines on the five benchmarks by at least 2.44%, 0.73%, 5.19%, 4.32%, and 0.24%, respectively. On average, it surpasses the baselines by 10.34%, 5.09%, 18.42%, 12.77%, and 6.11%, respectively. Our method outperforms the existing baselines primarily due to two reasons:

(1) Some methods do not consider the search for quantized models. For example, LAPT and RZ-NAS themselves do not include quantization, making them unable to leverage quantization to search for larger models. This limitation causes these methods to miss many larger models, leading to suboptimal performance on benchmarks that are highly quantization-friendly, such as Mhealth and Wharf. We conduct a statistical analysis of the optimal performance of the three modes across the five benchmarks, as shown in Figure 4. By analyzing the accuracy of the model architectures under the three modes, we found that in most cases, the origin mode performs better. However, for the Mhealth and Wharf benchmarks, the quantized mode tends to perform relatively better. Due to the characteristics of these two benchmarks, the performance of LAPT and RZ-NAS is inferior to that of LAPT(Quant) and RZ-NAS(Quant).

(2) The baselines generally lack suitable tools for evaluating models especially quantized models. In our experiments, we add quantization to the baselines, but the models they search for often perform worse than their original non-quantized versions. This is because incorporating quantization significantly enlarges the search space, and quantization can lead to varying degrees of performance degradation, making the search process much more challenging compared to non-quantized searches. In fact, to minimize the accuracy degradation caused by quantization as much as possible, we select six potential methods for static PTQ, and our Figure 5 illustrates the usage frequency of these six methods. By traversing

these six methods, we choose the best approach to reduce the negative impact of static PTQ. However, in the worst case, quantization may lead to an accuracy drop of over 25%. Therefore, without the design of suitable evaluation tools for origin and quantized models, these methods struggle to find better models within the quantized search space in the same time.

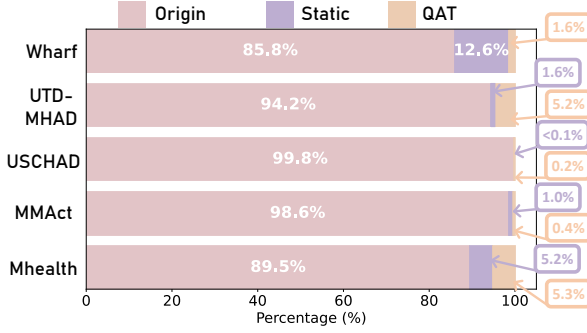


Figure 4: Quantization mode analysis. We record the mode with the highest accuracy for each model and calculate the proportion of the three modes that achieve the highest.

Additionally, we compare our method with the manually designed SOTA model UniMTS. From the experimental results, our method outperform this SOTA on the Mhealth and USCHAD benchmarks but does not surpass it on MMAAct, UTD-MHAD, and Wharf. We think this is acceptable, as UniMTS is a highly specialized, large-scale expert model exceeding 200MB, whereas our approach dynamically generates customized models, each of which is less than 8MB on edge devices.

5.3 Ablation Experiments

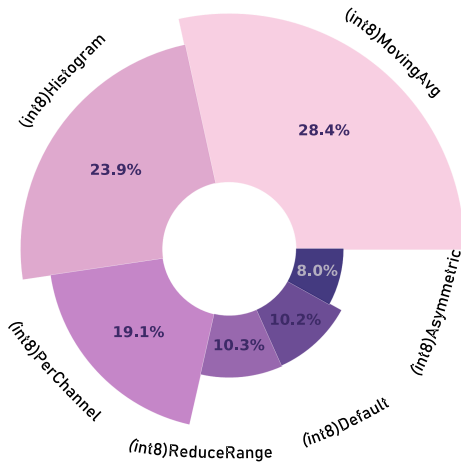


Figure 5: Static PTQ methods statistics.

In this ablation study, we demonstrate the effectiveness of the two components in our framework. To evaluate the effectiveness of the constraint validator, we remove this component and conduct

tests while keeping everything else unchanged. Similarly, to verify the effectiveness of the predictor, we replace it with a simple proxy composed of several training-free evaluation metrics, including memory, FLOPs, the balance between width and depth, Zen Score, SynFlow, and Grasp, multiplied by their respective weights.

From the experimental results, the accuracy of the two ablation methods is lower than that of EdgeGen in all cases. For ablation predictor, without the predictor, it cannot accurately estimate the performance of models, leading to missed opportunities for potential models. For ablation quantization, without the validator, it not only misses larger models but also lacks the ability to verify the integrity and validity of the models. Moreover, the validator provides suggestions based on violations. Besides, we find that ablation quantization consistently performs slightly worse than ablation predictor on the Mhealth and Wharf benchmarks. Similar to the behavior observed in several baselines from the 5.2, as Mhealth and Wharf are relatively quantization-friendly. On these benchmarks, the performance degradation caused by quantization is smaller, making it more likely for the quantized large models to outperform the smaller models.

5.4 Trade off between Time and Performance

As mentioned above, the predictor can quickly identify models with high accuracy. To support this claim, we conduct a trade-off experiment between time and performance. The detailed corpus, methods and metrics of the trade-off experiment are all shown in Section C.

Due to the extensive experimental data and high information entropy, the complete results are provided in Tables 9- Table 13 in the appendix C. Furthermore, to visually demonstrate how our predictor balances time cost and accuracy, we present simplified time-criterion graph in Figure 6.

As shown in the Figure 6, the time of EdgeGen’s predictor is less than 0.5 seconds. Furthermore, it demonstrates superior performance in both top-10 and Kendall metrics when applied to the original, Static, and QAT model variants. Rapid methods like Zico and SynFlow often show unstable and unsatisfactory performance, while training for 20 epochs gets relatively better accuracy but takes much longer (over 54.6 seconds), sometimes exceeding 100 seconds. Only the Predictor successfully balances both time efficiency and accuracy.

While the construction of the predictor introduces an initial computational overhead, its inference efficiency justifies this cost. Because our predictor achieves inference speeds 4.4× to 170× faster than zero-cost proxies (averaging 51×), which is also shown in Section C.

5.5 Foundation Model Experiments

To validate the transferability of EdgeGen, we replace the foundation model, switching from ChatGPT-4o to DeepSeek-V3. By replacing the foundation model, we test whether EdgeGen can still perform well under different generator conditions. Specific experimental results are shown in the Table 4. From the experimental results, EdgeGen outperforms all baselines across various constraints in all benchmarks. On average, EdgeGen outperforms the baseline methods with improvements of 2.15% on Mhealth, 2.16% on MMAAct,

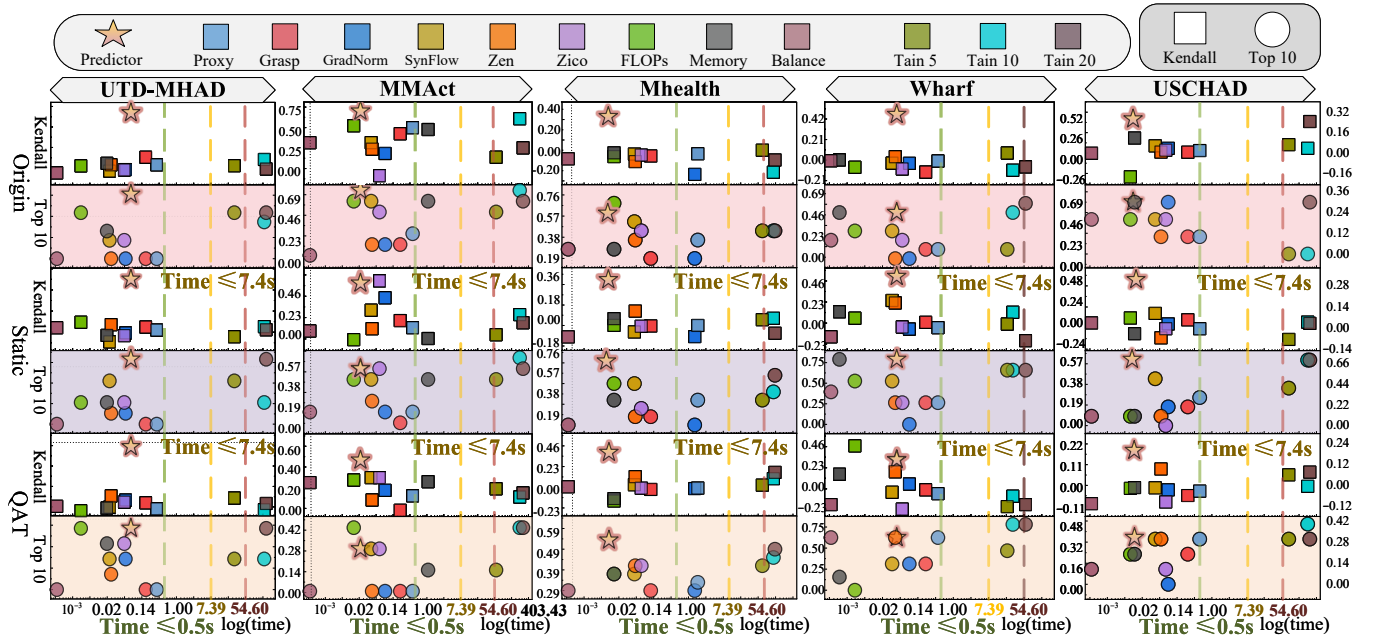
Table 3: Ablation study results across multiple benchmarks and memory constraints, including summary statistics.

Baseline\Benchmark	Mhealth			MMAct			UTD-MHAD		
	12MB	15MB	20MB	3MB	5MB	8MB	3MB	5MB	8MB
EdgeGen	89.63% QAT	96.95% QAT	99.39% Static	61.23% Static	62.07% Static	63.60% Origin	55.58% Origin	59.53% Origin	69.84% Origin
Ablation Predictor	89.27% QAT	87.98% Static	91.46% Origin	60.20% Static	60.04% Static	61.97% Origin	55.28% Origin	57.67% Origin	67.44% Origin
Ablation Quant	81.10% Origin	81.88% Origin	94.51% Origin	57.32% Origin	61.59% Origin	61.70% Origin	54.88% Origin	58.60% Origin	66.98% Origin
Baseline\Benchmark	Wharf			USCHAD			Summary		
	3MB	5MB	8MB	12MB	15MB	25MB	Average	Max	Min
EdgeGen	67.73% Static	67.05% Static	70.23% Static	72.35% Origin	72.71% Origin	77.47% Origin	74.16%	99.39%	55.58%
Ablation Predictor	58.64% Origin	64.55% Static	66.82% Static	69.73% Origin	71.87% Origin	73.90% Origin	68.19%	91.46%	55.28%
Ablation Quant	60.45% Origin	59.09% Origin	60.00% Origin	69.96% Origin	73.30% Origin	74.49% Origin	66.56%	94.51%	54.88%

Table 4: Results of all methods across multiple benchmarks and memory constraints with DeepSeek-V3.

Benchmark	Mhealth			MMAct			UTD-MHAD			Wharf			USCHAD		
Constraint	12MB	15MB	20MB	3MB	5MB	8MB	3MB	5MB	8MB	3MB	5MB	8MB	12MB	15MB	25MB
EdgeGen	93.90% QAT	98.35% Sta	99.78% Ori	65.17% Sta	59.62% Sta	61.49% Sta	57.42% Ori	72.56% Ori	73.95% Ori	71.36% Sta	74.09% Sta	73.64% Ori	72.11% Ori	72.71% Ori	73.50% Ori
LAPT	91.46% Ori	88.41% Ori	96.34% Ori	55.31% Ori	52.46% Ori	58.07% Ori	56.28% Ori	53.95% Ori	73.02% Ori	41.82% Ori	67.73% Ori	69.55% Ori	71.99% Ori	71.75% Ori	72.94% Ori
LAPT(Quant)	89.63% Sta	93.90% Sta	88.41% Sta	57.60% Sta	55.61% Sta	61.49% Ori	56.74% Ori	66.98% Ori	68.37% Ori	69.55% Sta	71.82% Sta	68.37% Sta	70.92% Ori	71.54% Ori	69.73% Ori
RZ-NAS	90.85% Ori	93.29% Ori	95.12% Ori	57.96% Ori	53.09% Ori	57.64% Ori	56.28% Ori	71.63% Ori	72.56% Ori	62.27% Ori	70.91% Ori	73.63% Ori	71.99% Ori	71.28% Ori	70.44% Ori
RZ-NAS(Quant)	86.59% Sta	97.17% QAT	96.95% Ori	58.81% Sta	59.57% Ori	61.43% Ori	43.72% QAT	66.51% Ori	73.89% Ori	68.64% Sta	67.27% Ori	65.00% Sta	66.74% Ori	56.26% QAT	69.73% Ori

Note: "Ori" refers to "Origin," and "Sta" refers to "Static."

**Figure 6: Performance comparison of EdgeGen's Predictor. On 5 benchmarks, we compare the top-10 and Kendall metrics with 12 baselines.**

0.56% on UTD-MHAD, 1.36% on Wharf, and 0.55% on USCHAD, demonstrating the transferability of our method.

6 Conclusion

In this work, we propose EdgeGen, a framework for dynamically searching lightweight and high-performance models within a quantized search space based on given constraints. This framework consists of a multi-beam MCTS algorithm, a constraint validator, and a performance predictor. It can identify high-quality original

models as well as outstanding quantized models after applying static PTQ or QAT.

7 Acknowledgments

We thank all reviewers for their constructive comments. This work is supported by the National Natural Science Foundation of China under grant No. 92582114 and No. 62272407, the Major Program of Zhejiang NSF Grant No. LD26F020002. Yi Gao is the corresponding author.

References

- [1] BANOS, O., GARCIA, R., HOLGADO-TERRIZA, J. A., DAMAS, M., POMARES, H., ROJAS, I., SAEZ, A., AND VILLALONGA, C. mhealthdroid: a novel framework for agile development of mobile health applications. In *International workshop on ambient assisted living* (2014), Springer, pp. 91–98.
- [2] BROWN, T. B., MANN, B., RYDER, N., SUBBIAH, M., KAPLAN, J., DHARIWAL, P., NEELAKANTAN, A., SHYAM, P., SASTRY, G., ASKELL, A., AGARWAL, S., HERBERT-VOSS, A., KRUEGER, G., HENIGHAN, T., CHILD, R., RAMESH, A., ZIEGLER, D. M., WU, J., WINTER, C., HESSE, C., CHEN, M., SIGLER, E., LITWIN, M., GRAY, S., CHES, B., CLARK, J., BERNER, C., MCCANDLISH, S., RADFORD, A., SUTSKEVER, I., AND AMODEI, D. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901. Presented at NeurIPS 2020.
- [3] CAI, H., GAN, C., WANG, T., ZHANG, Z., AND HAN, S. Once for all: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations* (2020).
- [4] CHAI, Y., BAILEY, L., JIN, Y., KARLE, M., AND KO, G. G. Bigger&faster: Two-stage neural architecture search for quantized transformer models. *CoRR* (2022).
- [5] CHEN, A., DOHAN, D., AND SO, D. Evoprompting: Language models for code-level neural architecture search. *Advances in neural information processing systems* 36 (2023), 7787–7817.
- [6] CHEN, H., WANG, Y., LIU, L., WEI, S., AND YIN, S. Hqnas: Auto cnn deployment framework for joint quantization and architecture search. *arXiv preprint arXiv:2210.08485* (2022).
- [7] JI, Z., ZHU, G., YUAN, C., AND HUANG, Y. RZ-nas: Enhancing llm-guided neural architecture search via reflective zero-cost strategy. In *Forty-second International Conference on Machine Learning*.
- [8] LIN, M., WANG, P., SUN, Z., CHEN, H., SUN, X., QIAN, Q., LI, H., AND JIN, R. Zen-nas: A zero-shot nas for high-performance image recognition. In *Proceedings of the IEEE/CVF international conference on computer vision* (2021), pp. 347–356.
- [9] LIU, F., TONG, X., YUAN, M., AND ZHANG, Q. Algorithm evolution using large language model. *arXiv preprint arXiv:2311.15249* (2023).
- [10] LIU, H., SIMONYAN, K., AND YANG, Y. Darts: Differentiable architecture search. In *Proceedings of the 7th International Conference on Learning Representations* (2019).
- [11] SHEN, L., YANG, Q., ZHENG, Y., AND LI, M. Autoiot: Llm-driven automated natural language programming for aiot applications. *arXiv preprint arXiv:2503.05346* (2025).
- [12] SHOAB, M., BOSCH, S., INCEL, O. D., SCHOLTEN, H., AND HAVINGA, P. J. Complex human activity recognition using smartphone and wrist-worn motion sensors. *Sensors* 16, 4 (2016), 426.
- [13] SRIDHAR, S. N., SZANKIN, M., CHEN, F., SUNDARESAN, S., AND SARAH, A. Simq-nas: Simultaneous quantization policy and neural architecture search. *arXiv preprint arXiv:2312.13301* (2023).
- [14] TAN, S., NAGARAJAN, T., AND GRAUMAN, K. Egodistill: Egocentric head motion distillation for efficient video understanding. *Advances in Neural Information Processing Systems* 36 (2023), 33485–33498.
- [15] TERMITTHIKUN, C., JAMTSO, Y., JEAMSAARD, J., MUNEEAWANG, P., AND LEE, I. Eeca-net: An early exit evolutionary neural architecture search. *Engineering Applications of Artificial Intelligence* 104 (2021), 104397.
- [16] XU, W., LI, Y., LIU, J., SUN, Y., CAO, Z., LI, Y., WEN, H., AND LIU, Y. Generative model for models: Rapid dnn customization for diverse tasks and resource constraints. *arXiv preprint arXiv:2308.15003* (2023).
- [17] ZEINATY, C. E., HAMDOUCHE, W., HERROU, G., MENARD, D., AND DEBBAH, M. Can llms revolutionize the design of explainable and efficient tinyml models? *arXiv preprint arXiv:2504.09685* (2025).
- [18] ZHANG, X., TENG, D., CHOWDHURY, R. R., LI, S., HONG, D., GUPTA, R., AND SHANG, J. Unimts: Unified pre-training for motion time series. *Advances in Neural Information Processing Systems* 37 (2024), 107469–107493.
- [19] ZHOU, X., WU, X., FENG, L., LU, Z., AND TAN, K. C. Design principle transfer in neural architecture search via large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2025), vol. 39, pp. 23000–23008.
- [20] ZOPH, B., AND LE, Q. V. Neural architecture search with reinforcement learning. In *Proceedings of the European Conference on Computer Vision* (2017).

A Benchmark Detail

To select appropriate benchmarks, we evaluate 18 common HAR benchmarks. We use a method that predicts the model by training for a certain number of epochs combined with a simple MCTS algorithm to search for the model. We obtain experimental results across these benchmarks, as summarized in Table 5.

Based on the experimental results shown in the Table 5, some benchmarks have already achieved accuracy exceeding 95%, indicating that these benchmarks are relatively easy and not suitable as

test benchmarks for experiments. Therefore, we first exclude benchmarks with excessively high accuracy due to their low difficulty, and then selected the five most representative benchmarks from the remaining ones: Mhealth, MMAct, USCHAD, UTD-MHAD, w-HAR, and Wharf.

The reason for choosing these five benchmarks is that their selection takes into account the characteristics of each benchmark, table 6 shows the detailed information of these five benchmarks. Since the HAR task is a time series multi-class classification task, the greater the number of classes, the more challenging the benchmark becomes. UTD-MHAD and MMAct have the largest number of classes among all 18 benchmarks, making them the two most difficult ones. Therefore, we included these two benchmarks in the experiments. However, it is worth noting that the time steps for these two benchmarks are relatively small, with one being 250 and the other 300. Both are relatively small datasets. To complement this, we added two more datasets, USCHAD and Mhealth, which are also challenging but have larger time steps. The time steps for these two datasets are 1000 and 500, respectively, requiring more memory. Finally, we add the Wharf benchmark, which also has relatively small time steps but very few channels. Although its number of classes is not as high as MMAct and UTD-MHAD, it has only 3 channels, meaning that each sample provides less information compared to other benchmarks, making it challenging.

These five benchmarks are carefully selected by us, and they are highly representative and challenging. The fact that EdgeGen outperforms other baselines and achieves good accuracy on these five benchmarks demonstrates the effectiveness and validity of our approach.

B Construction of Search Space

As shown in Table 1, our search space includes five common types of convolution (conv types).

The detailed information about these five convolution blocks types is displayed in Table 7. The structure of these five convolution blocks is shown in the table. The SE module we use is the SE (Squeeze-and-Excitation) module, and it is a lightweight architectural unit for recalibrating channel-wise feature responses in neural networks. It first applies global average pooling to squeeze spatial dimensions and generate channel-wise descriptors. Then, it uses two 1D convolution layers: the first reduces the channel dimension (squeeze), and the second restores it (excitation). A sigmoid activation scales the output to the range [0, 1], producing channel-wise weights. Finally, these weights are multiplied with the original input to emphasize important channels and suppress less relevant ones.

When generating model architectures in a generator, we provide the convolution block information from the search space to help it better utilize these convolution blocks.

C Trade off between Time and Performance

To demonstrate the ability of our predictor to accurately predict model performance in a relatively short time, we conducted a series of experiments. By comparing it with 12 baselines across 5 test data

Table 5: Experimental results of the simple search algorithm on 18 HAR benchmarks.

Benchmark																	
DSADS	70plus	Harth	Mhealth	MMAct	MotionSense	Oppg	PAMAP	realworld	Shoaib	TNDA-HAR	UCIHAR	USCHAD	ut-complex	UTD-MHAD	w-HAR	Wharf	WISDM
94.47%	99.20%	97.44%	68.29%	48.93%	98.13%	88.34%	94.76%	72.89%	89.20%	98.68%	93.18%	61.38%	94.64%	34.42%	68.85%	27.73%	66.40%

Table 6: Description of Human Activity Recognition benchmark.

Benchmark	Classes	Samples*Time Steps*Channels	Body Position	Activity Description
USCHAD	12	2236*1000*6	Right Front Hip	Walking (forward/left/right), running, upstairs, downstairs, jumping, sitting, standing, sleeping, elevator (up/down)
Mhealth	12	569*500*23	Chest, Right Wrist, Left Ankle	Standing, sitting, lying (back/side), walking, climbing stairs, bending forward, jumping, etc.
Wharf	14	1322*320*3	Wrist	Loading/unloading cargo, forklift operation, walking, standing, and other wharf operations
UTD-MHAD	27	861*300*6	Right Wrist or Right Thigh	Waving, punching, clapping, jumping, push-ups, and other actions + interactions (e.g., throwing/catching)
MMAct	35	7843*250*9	Trouser Pocket	Walking, running, cycling, driving, stationary (sitting/standing), transition activities

Table 7: Description of conv types.

Conv Type	SE Module	Skip Connection	Pointwise Conv	Depthwise Conv
DWSepConv	No	Yes	Yes	Yes
MBConv	Yes	Yes	Yes	Yes
DpConv	No	No	Yes	Yes
SeSepConv	Yes	No	Yes	Yes
SeDpConv	Yes	No	No	Yes

sets and evaluating 4 different metrics under three different quantization modes, we ultimately obtained comprehensive experimental comparison data for the predictor.

Corpus. In order to ensure the fairness of the experiment, test architectures are not added to the train set and validate set of the predictor training process. We select model architectures with 1 to 4 mixed stages from the 5 benchmarks including 3 modes.

Methods. These methods include the proxy score from RZ-NAS and five current SOTA model evaluation approaches: Grad Norm, Zen Score, SynFlow, ZiCo, and Grasp. In addition to these, we design three commonly used model performance evaluation metrics: FLOPs, memory usage, and the balance between width and depth. Furthermore, we incorporate the most straightforward evaluation methods, such as training the models for 5 epochs, 10 epochs and 20 epochs, into the comparison for a more comprehensive analysis.

Criterion. Since all evaluation methods aim to select the best model from multiple candidates, the key criterion is whether the ranking aligns with the true ranking based on model accuracy. Therefore, instead of using the Pearson correlation coefficient, we also include ranking-related metrics such as Kendall τ , Spearman ρ , and top- k ($k = 5, 10$) hit rates as evaluation indicators. Moreover, the time consumption of these methods is also a crucial factor to consider. Therefore, we measure the run time of all methods.

From Table 9 to Table 13, the prediction time of our predictor is consistently within 0.5 seconds, and it performs excellently across all four metrics. Even in cases where our method is not ranked first, it consistently remains in the top 3. In contrast, other methods with prediction times within 0.5 seconds fail to deliver stable superior performance, and their τ and ρ values frequently appear negative.

This is mainly because many of these quick prediction methods do not take the characteristics of quantization into account, and these methods are influenced by the model and the task.

Moreover, even methods such as train 5 epochs, train 10 epochs, and train 20 epochs often exhibit negative τ and ρ values. These two metrics are good indicators of ranking correlation, which means that using fewer training epochs to predict rankings can lead to severe misjudgments for models that converge more slowly.

Table 8: Different time limit results.

Baseline	Constraint	EdgeGen	LAPT	LAPT (quant)	RZ-NAS	RZ-NAS(quant)
Mhealth	20MB	99.99% Sta	94.51% Ori	85.98% Sta	95.73% Ori	92.68% Ori
MMAct	8MB	64.09% Ori	64.00% Ori	60.47% Ori	59.03% Ori	60.42% Ori
UTD-MHAD	8MB	74.66% Ori	73.95% Ori	66.98% Ori	64.19% Ori	73.02% Ori
Wharf	8MB	74.55% Sta	67.73% Ori	66.37% Ori	65.0% Ori	62.73% Sta
USCHAD	25MB	78.55% Ori	74.49% Ori	74.61% Ori	72.94% Ori	69.23% QAT

Note: "Ori" refers to "Origin," and "Sta" refers to "Static."

Worst of all, methods that rely on training to make predictions are very time-consuming, with all of them taking over 20 seconds. For larger data sets such as Mhealth and USCHAD, the prediction time for train 5 epochs exceeds 60 seconds, while train 20 epochs takes more than 140 seconds.

Predictor Overhead Analysis. While the construction of the predictor introduces an initial computational overhead, its inference efficiency justifies this cost. We utilized a server equipped with 4 RTX 4090 GPUs (4×24GB) and an Intel Xeon Gold 5317 CPU (125GB RAM). The data generation phase required approximately 120 hours, followed by 5 hours for predictor training. However, this is a one-time setup cost, and the trained predictor is reusable across different devices for the same task. Crucially, as detailed in Tables 9–13, our predictor achieves inference speeds 4.4× to 170× faster than zero-cost proxies (averaging 51×). Consequently, despite the initial overhead, the total time cost is significantly reduced for extensive architecture searches.

Table 9: Performance of 13 Methods under Three Quantization Modes (Origin, Static, QAT) on UTD-MHAD.

Mode	Metric	Methods												
		Predictor	Proxy	GradNorm	Zen	SynFlow	Zico	Grasp	FLOPs	memory	Balance	train(5)	train(10)	train(20)
Time		0.072s	0.32s	0.053s	0.023s	0.021s	0.049s	0.17s	0.004s	0.018s	$<10^{-4}$ s	28.36s	159.98s	179.06s
Origin	Top-5	0.4	0	0	0	0.2	0.2	0	0.2	0.2	0	0.2	0.4	0.4
	Top-10	0.7	0	0	0.2	0.2	0.2	0	0.5	0.3	0	0.5	0.4	0.5
	τ	0.674	0.038	-0.020	0.038	-0.038	-0.025	0.130	0.025	0.056	-0.060	0.027	0.103	-0.013
	ρ	0.847	0.058	-0.037	0.043	-0.061	-0.042	0.193	0.023	0.078	-0.084	0.051	0.155	-0.021
Static	Top-5	0.8	0	0.2	0	0.2	0.2	0	0.2	0	0	0.2	0.4	0.4
	Top-10	0.6	0	0.1	0.1	0.4	0.2	0	0.2	0.2	0	0.4	0.2	0.6
	τ	0.576	0.023	-0.008	0.082	-0.105	-0.037	0.056	0.108	-0.035	0.049	-0.049	0.061	0.027
	ρ	0.764	0.037	-0.009	0.122	-0.159	-0.051	0.084	0.161	-0.054	0.057	-0.075	0.095	0.046
QAT	Top-5	0.2	0	0.2	0	0.2	0.2	0	0	0	0	0.4	0.4	0
	Top-10	0.4	0	0.2	0.1	0.2	0.3	0	0.4	0.3	0	0.2	0.2	0.4
	τ	0.589	-0.019	0.066	0.107	-0.003	0.046	0.040	-0.038	-0.011	0.005	0.086	-0.028	0.033
	ρ	0.791	-0.027	0.111	0.164	-0.018	0.068	0.064	-0.066	-0.025	0.002	0.132	-0.034	0.033

Table 10: Performance of 13 Methods under Three Quantization Modes (Origin, Static, QAT) on Wharf.

Mode	Metric	Methods												
		Predictor	Proxy	GradNorm	Zen	SynFlow	Zico	Grasp	FLOPs	Memory	Balance	Train(5)	Train(10)	Train(20)
Time		0.041s	0.422s	0.084s	0.038s	0.032s	0.056s	0.211s	0.004s	0.002s	$<10^{-4}$ s	21.08s	28.83s	59.48s
Origin	Top-5	0.6	0.2	0	0.2	0.4	0.4	0	0.6	0.6	0	0	0.2	0.4
	Top-10	0.6	0.2	0.1	0.1	0.4	0.3	0.2	0.4	0.6	0.3	0.2	0.6	0.7
	τ	0.577	-0.012	-0.042	0.042	-0.039	-0.113	-0.153	-0.092	0.002	-0.008	0.089	-0.129	-0.086
	ρ	0.769	-0.049	-0.089	0.033	-0.041	-0.139	-0.223	-0.101	0.002	-0.014	0.126	-0.208	-0.128
Static	Top-5	0.8	0.2	0	0	0.6	0.4	0	0.4	0.6	0	0.2	0.6	0.4
	Top-10	0.6	0.2	0	0.2	0.4	0.2	0.2	0.4	0.6	0.3	0.5	0.5	0.5
	τ	0.531	-0.052	-0.069	0.230	0.257	-0.045	0.008	0.055	0.129	-0.156	-0.012	0.123	-0.200
	ρ	0.764	-0.085	-0.086	0.356	0.324	-0.064	-0.009	0.086	0.192	-0.214	-0.016	0.146	-0.288
QAT	Top-5	0	0.2	0.2	0.4	0	0	0.2	0	0	0	0.2	0.4	0.4
	Top-10	0.4	0.4	0.2	0.4	0.2	0.1	0.2	0	0.1	0.4	0.3	0.5	0.5
	τ	0.152	-0.025	0.025	0.089	-0.018	-0.106	-0.005	0.224	0.076	-0.082	-0.092	-0.039	-0.082
	ρ	0.230	-0.067	0.012	0.113	-0.002	-0.155	-0.016	0.333	0.150	-0.013	-0.146	-0.039	-0.138

Table 11: Performance of 13 Methods under Three Quantization Modes (Origin, Static, QAT) on Mhealth.

Mode	Metric	Methods												
		Predictor	Proxy	GradNorm	Zen	SynFlow	Zico	Grasp	FLOPs	Memory	Balance	Train(5)	Train(10)	Train(20)
Time		0.010s	1.702s	1.428s	0.047s	0.045s	0.066s	0.114s	0.014s	0.014s	0.0002s	69.49s	131.07s	143.518s
Origin	Top-5	0.6	0.2	0	0	0	0.4	0	0.4	0	0	0.2	0.2	0.4
	Top-10	0.5	0.2	0	0.2	0.4	0.3	0	0.6	0.1	0.1	0.3	0.3	0.3
	τ	0.457	0.069	-0.143	-0.011	0.066	0.055	0.046	0.039	0.078	0.017	0.105	-0.121	0.005
	ρ	0.636	0.099	-0.204	-0.012	0.099	0.081	0.085	0.075	0.115	0.017	0.146	-0.173	0.026
Static	Top-5	0.8	0.2	0	0	0.4	0.2	0	0.2	0	0	0.6	0.6	0.6
	Top-10	0.8	0.3	0	0.1	0.5	0.2	0.1	0.5	0.3	0	0.3	0.4	0.6
	τ	0.512	0.000	-0.125	0.162	-0.072	-0.009	-0.005	0.002	0.077	-0.128	0.065	0.081	-0.087
	ρ	0.678	-0.010	-0.168	0.243	-0.081	-0.012	-0.013	-0.006	0.114	-0.200	0.101	0.114	-0.147
QAT	Top-5	0.6	0.2	0	0.4	0	0.2	0	0.2	0	0	0	0.4	0.4
	Top-10	0.6	0.1	0	0.3	0.2	0.3	0	0.2	0.2	0	0.3	0.4	0.5
	τ	0.425	-0.009	-0.014	0.118	0.028	-0.011	-0.025	-0.153	-0.126	0.005	0.024	0.096	0.165
	ρ	0.590	-0.006	-0.019	0.158	0.049	-0.015	-0.042	-0.213	-0.200	0.021	0.042	0.133	0.242

Table 12: Performance of 13 Methods under Three Quantization Modes (Origin, Static, QAT) on USCHAD.

Mode	Metric	Methods												
		Predictor	Proxy	GradNorm	Zen	SynFlow	Zico	Grasp	FLOPs	Memory	Balance	Train(5)	Train(10)	Train(20)
Time		0.011s	0.496s	0.081s	0.054s	0.039s	0.071s	0.250s	0.009s	0.012s	0.0001s	82.18s	243.71s	275.07s
Origin	Top-5	0.4	0	0	0	0.4	0	0	0	0.2	0.2	0	0	0.2
	Top-10	0.3	0.1	0.3	0.1	0.2	0.2	0.1	0.2	0.3	0.2	0	0	0.3
	τ	0.259	0.020	0.037	0.008	0.056	0.022	0.007	-0.187	0.118	-0.002	0.066	0.037	0.247
	ρ	0.376	0.043	0.058	0.002	0.067	0.043	0.014	-0.254	0.185	0.012	0.076	0.073	0.358
Static	Top-5	0.6	0.2	0.2	0	0.4	0	0	0	0.2	0.2	0.4	0.4	0.6
	Top-10	0.7	0.3	0.2	0.1	0.5	0	0.2	0.1	0.1	0.1	0.4	0.7	0.7
	τ	0.338	-0.004	0.027	-0.065	0.097	-0.004	0.055	0.067	-0.037	0.034	-0.076	0.039	0.031
	ρ	0.471	-0.017	0.028	-0.087	0.155	-0.001	0.075	0.095	-0.068	0.057	-0.090	0.053	0.050
QAT	Top-5	0.4	0	0	0.4	0.2	0	0	0	0	0	0.4	0.4	0.4
	Top-10	0.3	0.3	0	0.3	0.3	0.1	0.2	0.2	0.2	0.1	0.3	0.4	0.3
	τ	0.199	-0.033	-0.026	0.095	-0.014	-0.096	-0.059	-0.018	-0.013	-0.106	0.061	-0.005	0.077
	ρ	0.303	-0.049	-0.027	0.144	-0.034	-0.125	-0.081	-0.030	-0.029	-0.159	0.095	-0.007	0.117

Table 13: Performance of 13 Methods under Three Quantization Modes (Origin, Static, QAT) on MMAct.

Mode	Metric	Methods												
		Predictor	Proxy	GradNorm	Zen	SynFlow	Zico	Grasp	FLOPs	Memory	Balance	Train(5)	Train(10)	Train(20)
Time		0.020s	0.430s	0.084s	0.039s	0.037s	0.061s	0.206s	0.013s	1.073s	0.0003s	59.59s	240.58s	291.92s
Origin	Top-5	0.2	0.2	0	0.2	0.2	0.2	0	0.4	0.4	0.2	0.4	0.6	0.6
	Top-10	0.8	0.4	0.3	0.3	0.7	0.6	0.3	0.7	0.7	0.2	0.6	0.8	0.7
	τ	0.361	0.200	-0.053	-0.011	0.053	-0.269	0.140	0.218	0.182	0.053	-0.090	0.287	0.002
	ρ	0.505	0.317	-0.084	-0.017	0.068	-0.345	0.181	0.324	0.245	0.088	-0.117	0.379	-0.023
Static	Top-5	0	0.2	0	0.2	0.2	0.6	0.2	0.4	0.4	0	0.4	0.4	0.4
	Top-10	0.6	0.2	0.2	0.3	0.5	0.6	0.1	0.5	0.5	0.2	0.5	0.7	0.6
	τ	0.291	-0.085	0.168	-0.094	0.062	0.310	-0.025	-0.186	-0.177	-0.113	-0.145	0.025	-0.044
	ρ	0.419	-0.095	0.224	-0.139	0.095	0.437	-0.061	-0.257	-0.266	-0.192	-0.178	0.064	-0.091
QAT	Top-5	0.4	0	0	0	0.4	0.4	0.2	0.4	0.2	0.2	0.2	0.4	0.6
	Top-10	0.5	0.3	0.3	0.3	0.5	0.5	0.3	0.6	0.4	0.3	0.4	0.6	0.6
	τ	0.306	-0.090	-0.030	-0.136	0.113	0.113	-0.251	0.085	0.067	0.057	-0.011	-0.103	-0.057
	ρ	0.428	-0.135	-0.067	-0.177	0.155	0.163	-0.315	0.110	0.096	0.102	-0.006	-0.147	-0.093

D Time Limit Experiments

To evaluate our algorithm’s performance, we also tested the results under different time constraints, as shown in the Table 8. Compared to the 1-hour time limit in the 5.2, we set a 2-hour time limit for this experiment. We select the maximum constraint condition for each benchmark to conduct the experiment.

As shown in Table 8, EdgeGen outperforms the other baselines. This demonstrates that our method can more quickly identify better models. We think this is mainly due to our validator’s ability to identify larger models, while multi-beam MCTS can balance exploring different models and utilizing the information from them. Most importantly, our predictor can quickly and accurately predict the performance of models. These factors enable EdgeGen to outperform other baselines.